

Folien-Analyse — NodeJS / serverseitig (Säule 5)

Quelle: nodejs/Day1 Http-Express-Npm · Day2 HTML-Generieren-Templates(+Servlets) · Day3 Streaming-Redirect-Scope-Patterns · Day4 Cookies-WebSockets-Persistenz. Gegen Node/Express-Docs + MDN verifiziert. Folienfehler mit . Themen-IDs N1-N8.

N1 — http-Modul, Server erstellen, npm (Produktion/Definition, hoch)

- **NodeJS:** alleinstehende JavaScript-Runtime (V8), serverseitig. Vorteile: asynchrone Performanz, JS+npm, plattformunabhängig. Nachteile: Code kann Server crashen, an JS gebunden.

```
const http = require('node:http');
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end("<h1>Hello</h1>"); // res.end() MUSS immer aufgerufen werden
});
server.listen(8000);
```

- **req (http.IncomingMessage):** `method`, `url`, `headers`, `httpVersion`; Events `data (chunk)/ end`.
- **res (http.ServerResponse):** `writeHead(status[,msg][,headers])`, `setHeader`, `write(chunk)`, `end(chunk)`.
- **Low-Level:** EIN Request-Listener `(req,res)=>{};` URL- und Methoden-Zuordnung muss selbst programmiert werden.
- **Start:** `node server.js`. Wichtig: nach Dateiänderung Server **stoppen + neu starten** (F5 im Browser reicht nicht).

Fallen / Bugs

- **Falle:** `res.end()` zwingend, sonst hängt die Antwort.
- **Falle:** http-Modul hat KEIN Routing — `req.url` / `req.method` selbst auswerten.

Drill

- [Erstellen] Minimaler http-Server auf Port 8000, der "Hi" sendet.

Lösung: `require('node:http').createServer((q,r)=>r.end("Hi")).listen(8000);`

N2 — Express: Routing, Middleware, req/res (Produktion, hoch)

- **Express:** High-Level npm-Modul, Quasi-Standard. `app.METHOD(PATH, HANDLER)`.

```
const express = require('express');
const app = express();
app.use(express.urlencoded({ extended: true })); // Body parsen
app.get('/', (req, res) => { res.send('Hello'); });
app.put('/user', (req, res) => { res.send('PUT /user'); });
app.listen(8000, () => console.log('up'));
```

- `res.send()` muss aufgerufen werden. `res.type('html')` setzt `text/html` + `charset utf-8`.

Middleware

- Funktionen `(req, res, next) => {}` mit Zugriff auf req/res und `next`. `next()` ruft nächste in der Kette.
- `app.use([path,] callback)`. Eigene Properties an `req` anhängen möglich (`req.requestTime = Date.now()`).

```
const myLogger = (req, res, next) => { console.log('LOGGED'); next(); };
app.use(myLogger);
```

Fallen / Bugs

- **Falle:** ohne `next()` bleibt die Request-Kette hängen.
- **Falle:** Reihenfolge der `app.use` /Routen zählt — Middleware muss VOR den Routen registriert sein.

Drill

- [Erstellen] Express-Route GET `/ping` → "pong".

Lösung: `app.get('/ping', (req, res) => res.send('pong'));`

- [Erstellen] Logger-Middleware, die "hit" loggt und weitergibt.

Lösung: `app.use((req, res, next) => {console.log('hit'); next();});`

N3 — HTML generieren / Templates / Servlets (Produktion, hoch)

Ansätze HTML zu erzeugen

Ansatz	Grundsprache	Code-Einbettung
String-Konkatenation (Node/Express)	JS	<code>response += "<h1>...</h1>"</code>
SSI (Server Side Includes)	HTML	<code><!--#echo var="..." --></code>
Java-Servlet	Java	<code>out.println("<h1>...</h1>")</code>
PHP	HTML	<code><?php echo \$_POST["x"]; ?></code>
ASP.NET (Razor)	HTML	<code>@{ ... } / @var</code>
JSP	HTML	<code><% out.println(...); %></code>
Template Engine (eta)	HTML	<code><%= it.x %></code>
* Pragmas geben "kein gültiges HTML" aus (Server rendert vor Auslieferung).		

Java-Servlets (Exkurs)

- Java-Klasse erbt von `HttpServlet` (heute `jakarta.servlet.http`), URL-Zuordnung via `@WebServlet("/pfad")` oder `web.xml` (Deployment-Descriptor). Methoden `doGet` / `doPost`. HTML via `response.getWriter().println(...)`. Braucht Servlet-Container (Tomcat, WildFly).

Express + Template-Engine (eta)

```
const eta = new Eta({ views: path.join(__dirname, "views") });
app.get("/", (req, res) => res.send(eta.render("hello-world", { name: "Eta" })));
```

- eta-Pragmas: `<% js %>` Code, `<%= ausdrück %>` escaped Ausgabe, `<%~ ausdrück %>` raw (XSS-Gefahr!), `<%~ include('./footer') %>` Partial. Spezialvariable `it` = gesamtes Parameterobjekt.
- `res.render(view[, locals][, cb])`, `app.set('views', ...)`, `app.set('view engine', ...)`.

Fallen / Bugs

- **Falle:** `<%~ %>` (raw) gibt unescaped aus → **XSS**; `<%= %>` escaped ist sicher.
- **Falle:** Logik und Darstellung nicht vermischen — Engine ist nur Darstellung.
- **Kontrast (Tabelle):** Bei Node/Servlet ist die **Programmiersprache** die Grundsprache (HTML wird ausgegeben); bei SSI/PHP/JSP ist **HTML** die Grundsprache (Code eingebettet).

Drill

- [Erstellen] eta-Template, das `it.name` escaped ausgibt.

Lösung: `<p>Hello <%= it.name %></p>`

- [MC] Welches eta-Pragma ist XSS-anfällig?

Lösung: `<%~ %>` (raw).

N4 — Streaming & Redirect (serverseitig) (Definition/Produktion, mittel)

Streamed HTML

- Browser rendert sobald erster Chunk da ist (Progressive Rendering, transparent). **Streamed HTML** = Server priorisiert/sendet Teile zuerst → muss selbst implementiert werden. `res.render` der Express-Engines unterstützt **KEIN** HTML-Streaming; Engine kann aber Einzelteile rendern und stückweise gesendet werden.

Redirect (HTTP)

- 3xx-Statuscodes, Antwort enthält `Location`-Header mit neuer URI.

Code	Bedeutung
301 Moved Permanently	diese + zukünftige Anfragen an neue URI
302 Found	nur jetzt neue URI, künftig wieder Original
307 Temporary Redirect	wie 302, HTTP-Methode bleibt gleich

```
res.redirect([status,] path); // Default-Status 302; path absolut oder host-relativ "/login"
```

Fallen / Bugs

- **Kontrast:** 301 permanent (Browser cached), 302 temporär, 307 temporär + Methode erhalten.

Drill

- [MC] Permanente Umleitung: welcher Statuscode?

Lösung: 301.

N5 — Gültigkeitsbereiche / Scope & Patterns (Tracing, mittel)

Variablen-Lebensdauer (5 Bereiche)

Je nachdem, wo eine Variable liegt, unterscheiden sich ihre Lebensdauer und Sichtbarkeit:

Bereich	Lebt
Template	nur während eines Render-Vorgangs (it, Schleifenvariablen)
Request	solange ein Request bearbeitet wird (req, res, lokale Vars)
Session	bis Session endet (selbst implementieren, z.B. Cookies)
Server	solange Server läuft (Konfig, Cache, Startparameter)
Persistent	über Server-Neustart hinaus (Datei/DB)

Model-1 vs. Model-2 (MVC)

Diese zwei Entwurfsmuster unterscheiden sich darin, ob Darstellung und Logik getrennt sind:

- **Model-1:** Aktion + Darstellung gemischt.
- **Model-2 (MVC):** Browser → Controller (verarbeitet Aktion) → View (generiert Antwort) → Browser. Trennung Aktion/Darstellung.

Redirect vs. Forward (Pattern)

Beide leiten eine Anfrage weiter - aber an unterschiedlichen Stellen und mit unterschiedlichen Folgen:

	Redirect (<code>res.redirect</code>)	Forward (<code>res.render</code>)
Ablauf	Server schickt 3xx, Browser lädt neue Seite (2. Request)	Server gibt direkt Inhalt zurück (intern)
URL im Browser	wechselt	bleibt stehen
Roundtrips	2	1

Fallen / Bugs

- **Kontrast:** Redirect = Client macht 2. Request, URL ändert sich; Forward = serverintern, URL bleibt.
- FOLIE: im Forward-Diagramm steht fälschlich "OK! (Redirect)" — Copy-Paste-Artefakt, gemeint ist Forward.

Drill

- [MC] URL bleibt gleich: Redirect oder Forward?

Lösung: Forward.

N6 — Cookies serverseitig (Produktion, hoch)

- **Problem:** HTTP ist zustandslos → ohne Mechanismus keine Sessions (Warenkorb, Login). **Lösung: Cookies.**
- **Ablauf:** (1) Browser fragt an → (2) Server antwortet mit `Set-Cookie` → (3) Browser hängt bei Folge-Requests auf gleicher Domain `Cookie:`-Header an.

```
Set-Cookie: sessionToken=abc123; Expires=Wed, 09 Jun 2021 10:18:14 GMT; Secure; HttpOnly
Cookie: theme=light; sessionToken=abc123
```

- **Anatomie:** `key=value` (+Attribute mit `;` getrennt). **Nur das key=value-Paar geht zurück**, Attribute nicht.

Attribut	Funktion
Domain/Path	Geltungsbereich
Expires/Max-Age	Geltungsdauer; ohne → Session-Cookie
Secure	nur über HTTPS übertragen
HttpOnly	nicht per JS (<code>document.cookie</code>) lesbar, nur HTTP(S)

Express

```
res.cookie(name, value [, {domain, expires, secure}]); // setzen
// lesen via Middleware:
const cookieParser = require('cookie-parser');
app.use(cookieParser());
app.get('/', (req,res)=> console.log(req.cookies)); // { sessionId: '...' }
```

- **Browser:** `document.cookie` (alle als `;`-String; einzeln von Hand parsen; setzen per Zuweisung).

Fallen / Bugs

- **Falle:** nur `key=value` wird gesendet — Attribute (Expires, Secure...) bleiben clientseitig.
- **Kontrast:** `Secure` (nur HTTPS) vs. `HttpOnly` (kein JS-Zugriff, Schutz vor XSS-Cookie-Diebstahl).

Drill

- [Erstellen] Express: Cookie `theme=dark` setzen.

Lösung: `res.cookie('theme', 'dark');`

- [MC] Welches Attribut blockiert JS-Zugriff auf das Cookie?

Lösung: `HttpOnly`.

N7 — WebSockets (Definition, niedrig)

- TCP-basiert, **bidirektional** (Server kann pushen), wenig Overhead (2–10 Byte). Verbindungsaufbau über HTTP (Upgrade), URL-Schema `ws` / `wss` (Port 80/443).

```
// Client:
const socket = new WebSocket('ws://localhost:8000/');
socket.onopen = () => {};
socket.onmessage = (event) => console.log(event.data);
socket.onclose = (event) => {};
socket.send("Hello Server");
// Server (npm "ws"): server.on('upgrade', ...) → wsSrv.handleUpgrade(...)
```

Fallen / Bugs

- **Kontrast:** HTTP = Request/Response, einseitig; WebSocket = persistente, bidirektionale Verbindung.
- **Falle:** Öffnen ist asynchron (`readyState` prüfen, `OPEN==1`); `onmessage` -Param ist Event, Daten in `event.data` .

Drill

- [MC] Welches Schema nutzt eine WebSocket-URL?

Lösung: `ws` / `wss` .

N8 — Persistenz (Datei/DB) (Definition/Produktion, mittel)

- **JSON-Datei requiren:** `const config = require("./persistent.json");` — nativ, versionierbar, aber **read-only** (Cache, ändert sich nicht zur Laufzeit).
- **Dateisystem** `fs` (Callback-, Promise-, Sync-API):
- `fs.readFile(path[, opts], (err,data)=>{})` (encoding 'utf8').
- `fs.writeFile(file, data[, opts], (err)=>{})` .
- Sicherheit: Server **NIEMALS als root/Admin** starten (Dateisystem-Rechte des Start-Users).
- **Datenbank:** npm-Bibliotheken (mysql, sqlite3, generisch Knex).

Fallen / Bugs

- **Falle:** `require('./x.json')` cached → Änderungen erst nach Neustart; für Schreiben `fs.writeFile` nutzen.

Drill

- [Erstellen] Datei "data.txt" mit Inhalt "hi" schreiben (fs).

Lösung: `fs.writeFile('data.txt', 'hi', err=>{});`